Microbiology LIMS POC — Process Reference (with Block Diagrams)

Visual companion to `microbio_lims_architecture.drawio` | 2026-04-29

What the system does

A subject-matter expert (SME) drops microbiology SOPs into S3. An agent pipeline, grounded in a hybrid retrieval substrate (BM25 + dense), reads them and emits two artifacts: **(A)** a Jira-ready Epic / Story tree, and **(B)** per-culture YAML configuration profiles (urine, blood, target-pathogen).

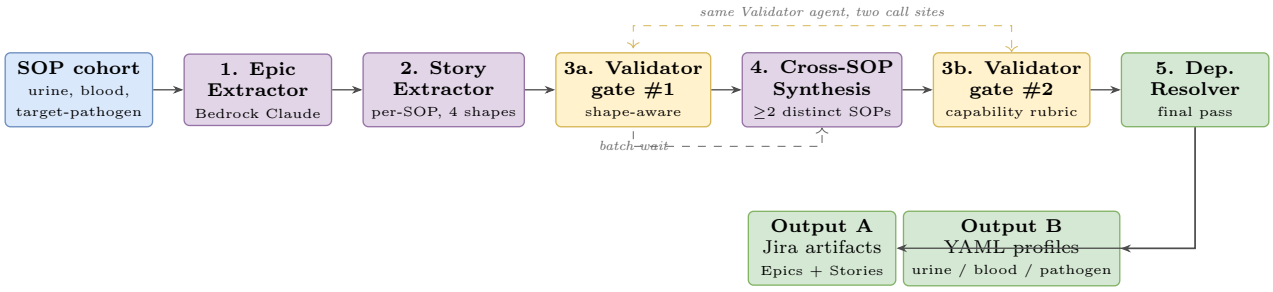
The Cytology Jira backlog is used as a *teaching corpus* (few-shot exemplars), not a registry to match against. The agent stops at Story — Tasks (test IDs, library versions, DB columns) are dev-authored once the team has codebase visibility.

Color legend: `src` source/data `cmp` compute `ai` AI/LLM agent `val` validator/decision `store` storage/output `xc` cross-cutting

Key terms (glossary)

SOP	Standard Operating Procedure — the source document the agent reads.
Epic	A capability domain; maps to a Jira Epic. E.g. *Microbiology Specimen Receipt*.
Story	A dev-actionable unit emitted by the agent. Each Story declares one of four `shape` values. Maps to a Jira Story.
Shape	Categorization assigned to each Story at extraction time. Drives Validator routing and downstream interpretation. Four values: `capability`, `workflow-stage-split`, `configuration-instance`, `cleanup`.
Theme (G1–G8)	Eight content tags spanning the LIMS workflow: G1 Pre-Analytic, G2 Analytic, G3 Post-Analytic, G4 Reporting, G5 QC, G6 Compliance, G7 Instrumentation, G8 Platform. Multi-label per chunk.
Teaching corpus	Curated SME-validated (SOP excerpt → Story) pairs from Cyto. Used as few-shot exemplars at extraction time.
Retrieval slot	A parallel retrieval channel scoped to a particular discipline + theme combination. Up to 5 slots fire per query.
Intent	Structured interpretation of a user query: {`action, themes, discipline, granularity, refs, confidence`}.
τ **(tau)**	Intent-confidence threshold (0.7). Below this, the system asks a clarifying question rather than retrieving.
θ **(theta)**	Retrieval-relevance threshold (0.5). If every slot falls below this, the system refuses to generate.
Batch wait	The pipeline pause between Validator gate #1 and Cross-SOP Synthesis. Synthesis cannot run until every per-SOP Story has cleared gate #1.

Agent pipeline — 5 agents, Validator gates twice



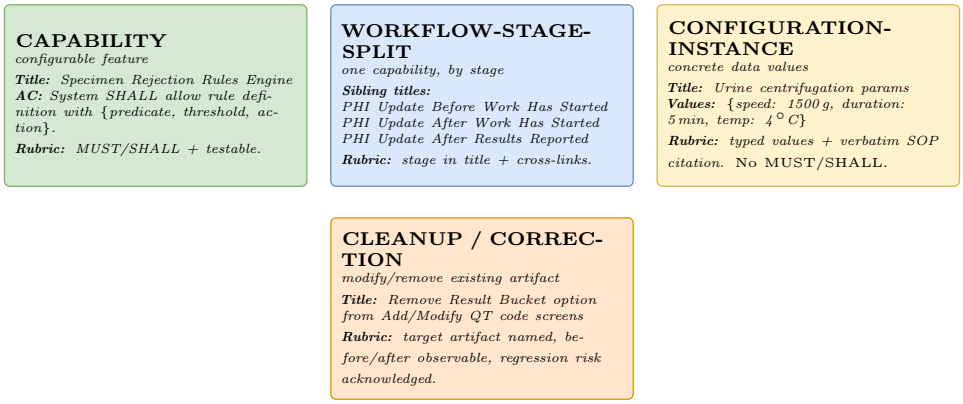
1. **Epic Extractor** — identifies capability domains. *Out:* Epic record per domain.
2. **Story Extractor** — per-SOP, multi-shape. Different elements of one SOP can take different shapes.
3. **Validator gate #1** — shape verification + shape-specific sub-rubric. Two revisions then SME-escalate.
4. **Cross-SOP Synthesis** — runs after gate #1 clears the entire batch. Behavioral-similarity clustering over (title + AC + source excerpt); cluster qualifies only if its members come from ≥ 2 *distinct* SOPs. Lifts a capability Story per cluster, with `cross_links[]` back to the contributing concretes.
5. **Validator gate #2** — same agent as 3a; capability rubric only. Synthesized stories are a *new* artifact, not a derivative, so they re-pass the bar.

6. **Dependency Resolver** — story↔story dependencies, stage-split sibling links, capability↔child links. Finalizes the artifact graph for Jira push.

Four story shapes — what "shape" means

A **shape** is the categorization the Story Extractor assigns to each Story. It tells the Validator which sub-rubric to apply, and tells downstream consumers (devs, the YAML emitter) how to interpret the AC. Per-SOP extraction picks one of four shapes per element. Cross-SOP synthesis only ever emits the `capability` shape.

shape	Use when...	Validator rubric
`capability`	SOP describes a configurable system feature you can imagine multiple labs/cultures parameterizing	MUST/SHALL on AC; testable; configurability boundaries explicit
`workflow-stage-split`	Same capability, but SOP describes different behavior at different workflow stages (before / during / after results)	Stage explicit in title; sibling stories enumerated and cross-linked
`configuration-instance`	Concrete data values for one culture / lab. *No new feature — just data.*	Typed values + verbatim citation. *No MUST/SHALL.*
`cleanup`	Modify or remove an existing artifact. *No new feature.*	Target named + before/after observable + regression risk



Schemas — deep dive

Epic schema

```
{ id: "EPIC-MICRO-001", title: "Microbiology Specimen Receipt",
  description: "...", discipline: "microbiology",
  themes: ["G1","G5"], cyto_epic_analog: "EPIC-CYTO-007" }
```

field	type	req'd	notes
`id`	string	yes	Jira-shaped Epic ID.
`title`	string	yes	Short capability-domain name.
`description`	string	yes	1–3 sentence scope statement.
`discipline`	enum	yes	`microbiology` | `histology` | `cytology`.
`themes[]`	array ⟨G1..G8⟩	yes	Multi-label; an Epic can span themes.
`cyto_epic_analog`	string	no	Cyto Epic ID this Epic is structurally analogous to (D13).

Story schema

```
{ id: "STORY-MICRO-1042", epic_id: "EPIC-MICRO-001", shape: "capability",
  title: "Configurable specimen rejection rules engine", description: "...",
  acceptance_criteria: [{when, then, expected_value?}, ...],
  source_citations: [{doc_id, section, line_range}, ...],
  dependencies: ["STORY-MICRO-1018"],
```

```
  cross_links: ["STORY-MICRO-1043","STORY-MICRO-1044","STORY-MICRO-1045"],
  estimated_complexity: "M", edge_cases_handled: [...],
  status: "validated", source_chunks: ["chunk_001","chunk_017"] }
```

field	type	req'd	notes
`id`	string	yes	Jira-shaped Story ID.
`epic_id`	string	yes	FK to parent Epic.
`shape`	enum (4)	yes	Drives Validator routing.
`title`	string	yes	Concrete + unambiguous; for stage-split, must name the stage.
`description`	string	yes	User-story format for capability/stage-split; descriptive prose for config-instance/cleanup.
`acceptance_criteria[]`	array	yes	Capability/stage-split: MUST/SHALL phrasing. Config-instance: `expected_value` populated. Cleanup: before/after wording.
`source_citations[]`	array ⟨{doc_id, section, line_range}⟩	yes	Must resolve to indexed chunks; verbatim excerpt for config-instance.
`dependencies[]`	array ⟨Story ID⟩	no	Stories that must complete before this one is workable.
`cross_links[]`	array ⟨Story ID⟩	no	Capability ↔ concrete contributors; stage-split sibling group. Backbone of the Story DAG.
`technical_hints`	string	no	Dev-facing guidance.
`estimated_complexity`	enum `S`|`M`|`L`	yes	Heuristic, sprint sizing.
`edge_cases_handled[]`	array of strings	no	Surfaced corner cases.
`status`	enum	yes	`draft` | `validated` | `ready`.
`source_chunks[]`	array ⟨chunk ID⟩	yes	Audit trail to the retrieval substrate.

Cluster schema (intermediate, in Cross-SOP Synthesis)

Not persisted to Jira; lives only between gate #1 and gate #2 in memory.

```
{ cluster_id: "clust_rejection_3sop",
  member_story_ids: ["STORY-MICRO-1043","STORY-MICRO-1044","STORY-MICRO-1045"],
  source_sops: ["doc_micro_014_v3","doc_micro_022_v1","doc_micro_037_v2"],
  centroid_summary: "Reject specimen on a culture-specific predicate.",
  proposed_capability_title: "Configurable specimen rejection rules engine" }
```

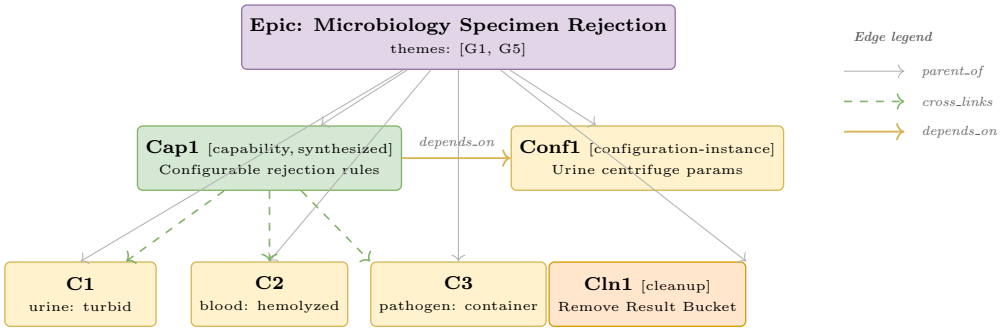
A cluster qualifies for lifting **iff** `len(unique(source_sops)) >= 2`. Two stories from the same SOP do not qualify, even if they cluster tightly.

Validator output schema (when a Story fails)

```
{ story_id: "STORY-MICRO-1042", verdict: "revise",
  shape_declared: "capability", shape_verified: "capability",
  failed_checks: [
    {rule:"must_shall_present", severity:"error", message:"AC 1 lacks MUST/SHALL phrasing."},
    {rule:"ambiguous_quantifier", severity:"error", message:"AC 2 uses 'appropriate'."}
  ], attempt: 1 }
```

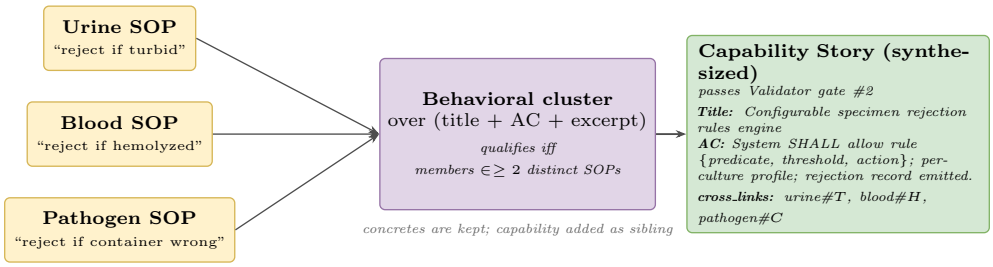
Story relationships (graph view)

The Story schema's `dependencies[]` and `cross_links[]` arrays serialize a directed acyclic graph. Walking it answers "what's blocked by X?" or "if I deprecate the synthesized capability story, which concretes need merging?"



edge	direction	meaning
`parent_of`	Epic → Story	Implicit Jira hierarchy (encoded in `epic_id`).
`cross_links`	Story ↔ Story	Bidirectional association. Capability ↔ concrete (synthesis); stage-split sibling sets.
`depends_on`	Story → Story	Directed prerequisite. The Dependency Resolver topologically orders the backlog for sprint planning.

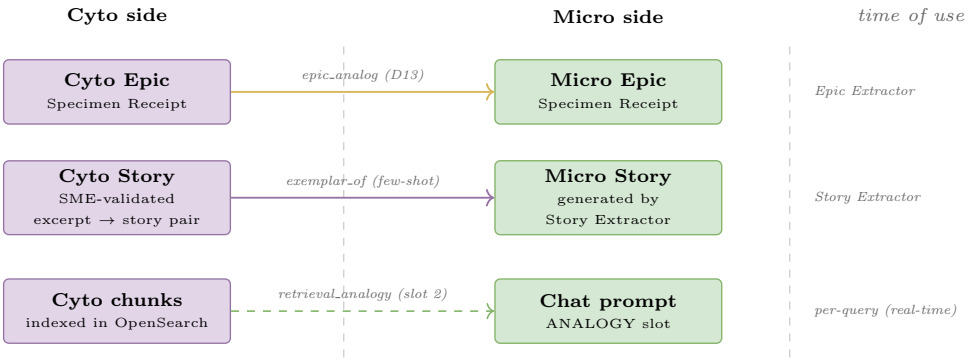
Cross-SOP synthesis — recurrence becomes capability



The recurrence threshold (≥ 2 distinct SOPs) is tunable. With three SOPs in the POC, this means anything seen in two of three lifts. The synthesized capability story passes Validator gate #2 before persistence.

Cyto → Micro lineage (graph view)

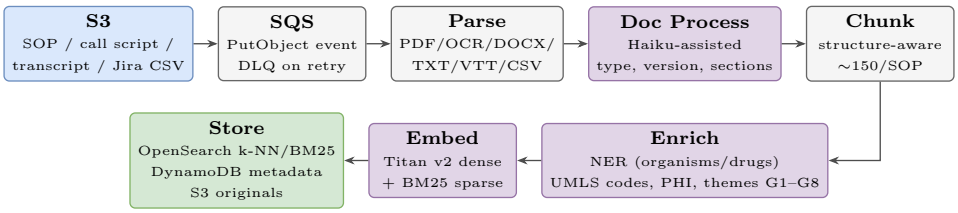
Three distinct mechanisms transfer information from Cyto to Micro. They operate at different times in the system's life cycle.



mechanism	direction	when it fires / what flows
epic_analog	Cyto Epic → Micro Epic	At Epic-Extractor time. Just an ID reference — recorded for SME traceability (D13).
exemplar_of	Cyto Story → Micro Story	At Story-Extractor time. The Cyto (excerpt → Story) pair lands in the prompt as an in-context example.
retrieval_analogy	Cyto chunks → chat prompt	Per-query, real-time. Retrieved Cyto excerpts populate the ANALOGY context section. *Adapt, don't copy.*

The same Cyto material thus serves three roles: a curated few-shot teaching set, an indexed corpus for live retrieval, and a structural reference at the Epic level.

Ingestion substrate — S3 to indexed chunks



I/O trace — one document through the pipeline (SOP-MICRO-014_Gram-Stain_v3.docx):

```
[parse out]
text: 28,420 chars / 18 pages
format: docx, ocr_used: false, parse_ms: 320
```

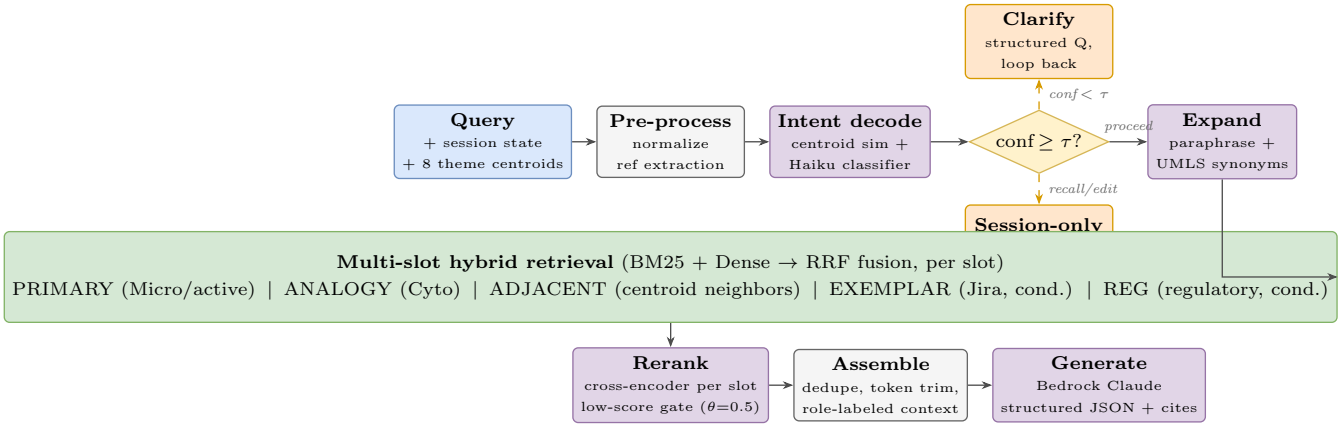
```
[doc process out]
doc_id: doc_micro_014_v3_abc12
discipline: Microbiology, version: v3
section_map: {Purpose:[200,450], Procedure:[721,2400],
AC:[2401,3100], QC:[3101,3650]}
```

```
[enrich out, chunk_001]
entities: {organisms:["Gram-pos cocci"],
specimens:["sputum"], codes:["87209"]}
phi_detected: false
themes: {G2: 0.84, G5: 0.41}
```

```
[embed out, chunk_001]
dense_vector: [0.0234, -0.1849, ..., 0.2891] (1024-d, Bedrock Titan v2)
bm25_tokens_enriched: ["slide","preparation","87209","smear","primary","source",...]
```

```
[store, sinks written]
PUT chunk_001 → OpenSearch k-NN index (vector)
PUT chunk_001 → OpenSearch BM25 index (sparse)
PUT chunk_001 → DynamoDB chunk_metadata (entities, themes, phi_status, version)
```

Retrieval substrate — per query, real-time



I/O trace — one query: "Generate epics for specimen rejection in microbiology"

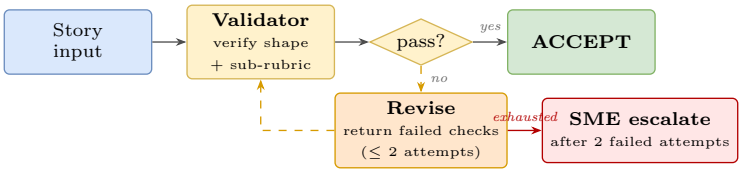
```
[intent decode out]
centroid_dist: {G1: 0.81, G5: 0.18}
classifier: {action: generate, themes: [G1, G5], discipline: Micro, granularity: epic,
refs: [], conf: 0.91}
decision: PROCEED (τ=0.7, 0.91 ≥ τ)
```

```
[retrieve out] 48 candidate chunks
  slot_1 PRIMARY (Micro/G1+G5): 20
  slot_2 ANALOGY (Cyto/G1+G5): 10
  slot_3 ADJACENT (Micro/G2 neighbors): 10
  slot_4 EXEMPLAR (Jira, generate∧epic): 5
  slot_5 REG (CLSI on G5): 3
```

```
[rerank + assemble out]
trim: 8/4/3/3/2 = 20 chunks (all slots ≥ θ=0.5)
prompt sections: PRIMARY, ANALOGY, ADJACENT, EXEMPLAR, REG
+ session drafts + output schema
total tokens: 6,840
```

```
[generate out] structured JSON
{ epic_id: epic_draft_3,
  title: "Specimen Rejection Workflow",
  theme: G1,
  citations: ["s3://.../SOP-MICRO-007 §AC",
              "s3://.../SOP-CYTO-021 §Procedure",
              "CLSI M22 §3.4"],
  stories: ["Validate transport temp", ...] }
```

Validator — shape-aware revise/escalate flow



Sub-rubrics by shape (cheat sheet):

- **Capability:** MUST/SHALL on every AC; testable; no ambiguous quantifiers ("appropriate", "relevant", "necessary"); configurability boundaries called out; SOP citation present and resolves; complexity S/M/L; edge cases enumerated.
- **Stage-split:** stage explicit in title; entry/exit observable per stage; siblings enumerated and cross-linked; transition contract clear; boundary cases handled.
- **Configuration-instance:** typed concrete values (units / enums / nullable rules); target table or profile named (e.g. `cultures/urine.yaml#centrifugation`); SOP excerpt cited verbatim; scope (culture/lab) stated. *MUST/SHALL not required.*
- **Cleanup:** target artifact named (id, screen path, file path); before/after observable; regression risk acknowledged; DoD includes a smoke test or QA path. *MUST/SHALL not required.*

What ships — Output A (Jira) and Output B (YAML)

Output A — Jira artifacts. The connected Epic / Story DAG (graph above), pushed to Jira via REST. The graph is what the dev team works against; the YAML profile (Output B) is what their code consumes at runtime.

Output B — Per-culture YAML configuration profile. One YAML file per culture in scope. Carries the concrete typed values that `configuration-instance` Stories' AC referenced. Profiles are versioned, dated, and cite back to the SOPs they draw from. **The Story body never duplicates these values** — it references them by `related_story_ac`, so a single source of truth is preserved.

Full profile structure (`cultures/urine.yaml`)

```yaml
profile:
  culture: urine          # urine | blood | target_pathogen
  version: 3
  effective_date: 2026-04-15
  source_sops: [doc_micro_014_v3, doc_micro_022_v1]

pre_analytic:
  collection:
    container: {value: "sterile cup", units: enum, citation: "doc_micro_014_v3 §1.2 L8"}
    volume_min: {value: 10, units: mL, citation: "doc_micro_014_v3 §1.2 L11"}
  transport:
    temp_range: {min: 2, max: 8, units: degC, citation: "doc_micro_014_v3 §1.4 L18-19"}
    time_max: {value: 4, units: h, citation: "doc_micro_014_v3 §1.4 L20"}
  rejection_predicates:
    - id: REJ_TURBID
      condition: "turbidity > threshold"
      threshold: {value: 200, units: NTU}
      action: reject
      citation: "doc_micro_014_v3 §1.5 L24"

analytic:
  centrifugation:
    speed: {value: 1500, units: g, citation: "doc_micro_014_v3 §3.2 L41-43"}
    duration: {value: 5, units: min, citation: "doc_micro_014_v3 §3.2 L44"}
    temp: {value: 4, units: degC, citation: "doc_micro_014_v3 §3.2 L45"}
  inoculation:
    media:
      - {type: BAP, volume_uL: 10, citation: "doc_micro_014_v3 §4.1 L52"}
      - {type: MAC, volume_uL: 10, citation: "doc_micro_014_v3 §4.1 L53"}
```

```
    incubation: {temp: 35, atmosphere: "5% CO2", duration_h: 18,
        citation: "doc_micro_014_v3 §4.3 L60"}

post_analytic:
  reporting:
    cfu_threshold: {value: 100000, units: CFU_per_mL,
        citation: "doc_micro_014_v3 §6.1 L88"}

related_story_acs: # back-references to Stories that consume this profile
  - STORY-MICRO-1042#AC-2 # Configurable rejection rules engine
  - STORY-MICRO-1077#AC-1 # Centrifugation step
```

How a Story's AC references the profile

A `configuration-instance` Story's AC carries the citation; the profile is what gets read by code. Example — `STORY-MICRO-1077`:

```
acceptance_criteria:
  - when: "urine profile loaded"
    then: "centrifuge step uses cultures/urine.yaml#analytic.centrifugation"
    expected_value: "speed=1500g, duration=5min, temp=4degC"
```

Where to put a value: Story body vs. profile

If the value is...	Put it in...
The same across cultures and parameterizable in the rules engine	Story body as parameter spec (capability shape)
Different per culture / per lab	YAML profile (configuration-instance shape)
A literal constant in the rejection logic	Profile, with the Story's AC pointing at it

Cross-cutting

KMS encryption at rest and in transit; IAM least-privilege with VPC endpoints (Bedrock calls stay in-VPC); DLQ for parse / OCR / embed failures; CloudWatch metrics and alarms (e.g. OCR fail-rate > 5%); CloudTrail audit log on every API call; idempotent ingestion via content-hash dedupe.

Generalization: when Histology arrives, the same agents and the same retrieval substrate run — only the exemplar corpus and config priors get extended. No model retraining.